

Particle Filter vs. Extended Kalman Filter Localization for a Mobile Manipulator In ROS2 and Gazebo

Final Project DRAFT

Jacob Taylor Cassady

605.745: Reasoning Under Uncertainty
Johns Hopkins University, Whiting School of Engineering

November 1, 2026



Contents

1 Overview **3**

2 Problem Statement **5**
2.1 Kidnapping . . . 5
2.2 Bayes Filter . . . 5
2.3 Algorithm Overview . . . 6

3 Study Approach **7**
3.1 System Model . . . 7
3.1.1 Platform & Sensors . . . 7
3.1.2 Unicycle Kinematics . . . 8
3.1.3 Odometry Noise Model . . . 9
3.1.4 Map . . . 9
3.2 Dead Reckoning . . . 9
3.2.1 Prediction . . . 10
3.3 Extended Kalman Filter (EKF) . . . 10
3.3.1 Augmented Motion Model . . . 10
3.3.2 Prediction . . . 12
3.3.3 Measurement Update . . . 12
3.4 Particle Filter . . . 13
3.4.1 Prediction . . . 14
3.4.2 Measurement Update . . . 14
3.4.3 Resampling . . . 15
3.5 Kidnapping . . . 15
3.6 Metric Collection . . . 15

4 Results **16**
4.1 Algorithm Accuracy . . . 16
4.2 Runtime Analysis . . . 16
4.3 Kidnap Recovery . . . 16
4.4 Particle Count Analysis . . . 16

5 Conclusion **17**

6 Acknowledgements **18**

7 Appendix **19**
7.1 Base Localization Node Implementation . . . 19
7.2 Dead Reckoning Node Implementation . . . 23
7.3 Extended Kalman Filter Node Implementation . . . 26
7.4 Particle Filter Node Implementation . . . 36

1 Overview

A critical precursor to robot path planning is the ability to determine a robot's pose relative to a known environment. The problem of estimating a robot's pose relative to a known map from proprioceptive measurements and exteroceptive measurements is referred to as localization [1]. Without a reliable estimate of a robot's state with respect to the world, a robot cannot safely interact with its surroundings. This paper presents a study comparing algorithms for performing localization of a differential drive mobile manipulator with a two-dimensional (2-D) Light Detection and Ranging (LiDAR) sensor and an odometer. Each algorithm approximates a probability distribution for the pose, called the *belief*.

To facilitate the development of the algorithms, this study utilizes the Robotic Operating System 2 (ROS2) [2]. ROS2 is a middleware framework that is built on top of the Data Distribution Service (DDS) protocol [3] to support distributed architectures. Within the ecosystem, independent processes are referred to as "nodes". Nodes communicate with one another by publishing and subscribing to shared message queues known as "topics." The distributed architecture enables asynchronous processing of sensor data and commands.

Additionally, this study will make use of the Gazebo simulation environment, which provides a high fidelity physics engine and sensor simulation plugins. The experiments will take place in a simulated factory world populated with obstacles and goal points for navigation. Figure 1 illustrates the factory layout, goal points, and the initial position of the robot in 2-D. Figure 2 provides a screenshot of the factory at runtime in three dimensions (3-D).

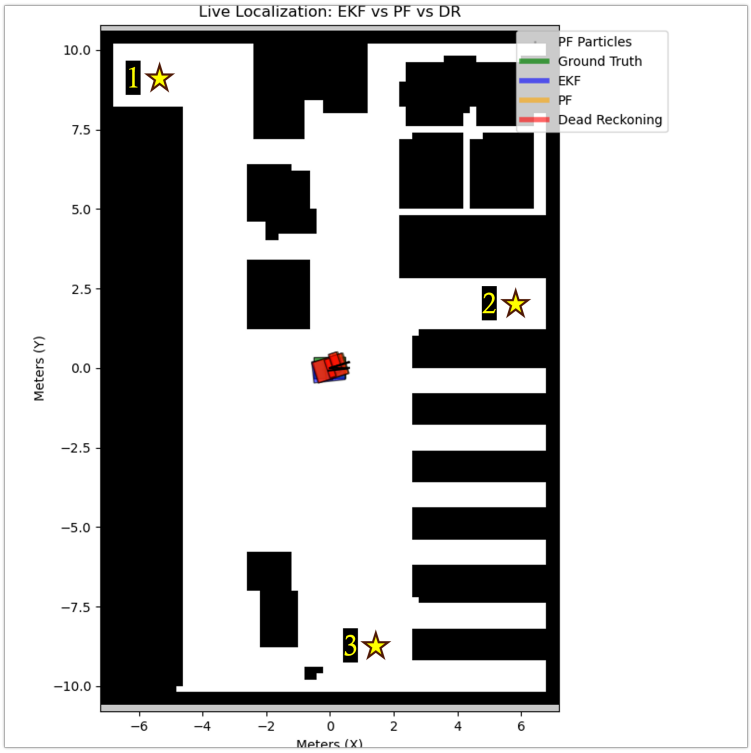


Figure 1: 2-D Rendering of the factory floor, obstacles, and designated goal points.



Figure 2: Gazebo factory 3-D simulation.

The primary objective of this study is to evaluate and compare the performance of different localization algorithms when deployed on a mobile manipulator in a factory setting. Specifically, this work investigates the trade-offs between Dead Reckoning, Particle Filter, and Extended Kalman Filter approaches. By analyzing localization accuracy, computational efficiency, and robustness to sensor noise, this study aims to provide practical insights into selecting the appropriate state estimation strategy for autonomous mobile manipulation.

2 Problem Statement

In mobile robotics, the planning environment is often projected onto a two dimensional plane. Let the pose of the robot at time t be defined by the state vector $\mathbf{x}_t = [x_t, y_t, \theta_t]^T$, where x_t and y_t represent the planar coordinates and θ_t represents the heading. As the robot moves, its true state transitions according to a non-linear motion model:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t) + \mathbf{w}_t \tag{1}$$

where $\mathbf{u}_t$ is the control input (such as translational velocity v_t and rotational velocity ω_t) and $\mathbf{w}_t$ represents process noise caused by factors like wheel slip or mechanical imperfections.

To correct the drift induced by the process noise, the robot relies on exteroceptive sensors, modeled as:

$$\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t \tag{2}$$

where $\mathbf{z}_t$ is the sensor measurement (such as a LiDAR scan) and $\mathbf{v}_t$ represents the measurement noise. The core problem of localization is to estimate the *belief*, or the probability distribution over the state x_t at time t, conditioned on all past measurements $z_{1:t}$ and all past controls $_{1:t}$[1].

$$bel_{x_t} = p(x_t \mid z_{1:t}, u_{1:t}) \tag{3}$$

2.1 Kidnapping

While tracking a robot's pose under normal operating conditions is well understood, catastrophic localization failures remain a significant challenge. The most extreme form of this failure is known as the "kidnapped robot problem" [4]. This occurs when a robot is teleported to a different location without its odometry registering the movement, completely invalidating its current belief state.

In the real world, this could happen if a robot loses power and is re-powered at a different position or if the robot being tracked loses connection with a tracking hub and then reconnects at a different time. To recover from a kidnapping event, a localization algorithm must be capable of abandoning its incorrect localized belief and search the map to find its new true pose.

2.2 Bayes Filter

The most general algorithm for calculating beliefs, shown in Equation 3, is the **Bayes filter**[1]. The Bayes filter is recursive and calculates the belief $bel(x_t)$ at time t from the $bel(x_{t-1})$ at time $t-1$. The Bayes filter has two steps: prediction and measurement update. The prediction calculates the belief $\overline{bel}(x_t)$ over x_t based on the prior belief over x_{t-1} and u_t as shown in Equation 4.

$$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1}) bel(x_{t-1})\, dx \tag{4}$$

The measurement update calculates the final belief $bel(x_t)$ by multiplying the belief $\overline{bel}(x_t)$ by the probability that the measurement z_t was observed as shown in Equation 5.

$$bel(x_t) = \eta p(z_t \mid x_t) \overline{bel}(x_t) \tag{5}$$

2.3 Algorithm Overview

Various algorithms exist to solve the localization problem. The three algorithms selected for this study have different levels of robustness for kidnapping as well as different expected accuracies and computational costs. They are all based on the Bayes filter with increasing complexity.

The first algorithm, **Dead Reckoning**, is the most basic form of state estimation. It relies only on integrating the control inputs u_t over time to estimate the current pose [5]. The Dead Reckoning algorithm implements only the prediction step of the Bayes filter. Because Dead Reckoning lacks a measurement update step, process noise accumulates with time, making the predicted pose stray from truth with time. This algorithm is used as a comparative baseline for the other two algorithms.

The second algorithm, **Extended Kalman Filter (EKF)**, tackles the non-linear motion and measurement models by linearizing them via Taylor series expansion and computing the Jacobians of f and h from a kinematic model and sensor model [1]. The EKF includes both the prediction step and the measurement update step described by the Bayes filter. Both Dead Reckoning and EKF are considered *Gaussian Filters* where the probability density over x_t can be characterized by the mean μ and the covariance Σ. Gaussians are unimodal, meaning the posses a single maximum and perform poorly in problems which many distinct hypotheses exist.

The third algorithm, **Particle Filter (PF)**, is a non-parametric approach that represents the belief state using finite sets of weighted random samples called particles [6]. Because the PF does not assume a single Gaussian distribution as the EKF does, the PF can represent complex, multi-modal beliefs.

This study evaluates these algorithms under various factory conditions to test two primary hypotheses. First, it is hypothesized that the Particle Filter will recover localization accuracy following a kidnapping event significantly more reliably than the Extended Kalman Filter. This is due to the Particle Filter's ability to maintain multiple pose hypotheses simultaneously. Second, it is hypothesized that given a well-defined Gaussian initial pose prior (where the robot's starting location is known), the Extended Kalman Filter will achieve localization accuracy comparable to or better than the Particle Filter while requiring a significantly lower computational cost.

3 Study Approach

This section begins by outlining the model that the three algorithms share including the mobile platform and its sensors, the kinematic model and its Jacobians, the odometry noise model, and the map representations. Then it describes the algorithms and their implementation specifics. At the end, this section describes the mechanisms for kidnapping in the Gazebo environment and the metrics to be collected in the experiments presented in section 4.

3.1 System Model

A high level overview of the ROS2 system used in this study is illustrated in Figure 3. The algorithms under test represent the `AgentLocalizer` box in gold. It receives sensor data from Gazebo plugins, described below. Additionally, the `AgentLocalizer` subscribes to a map representation generated from an octomap_server node. The `AgentLocalizer` publishes its pose to a local motion planner and a position controller. The position controller then sends commands to a Gazebo DiffDrive [7] plugin which sends joint velocities to the simulated robot's wheels.

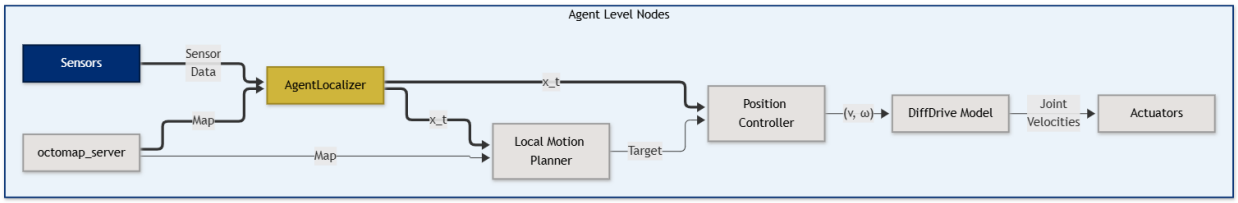


Figure 3: Full ROS2 system diagram

The Local Motion Planner and Position Controller nodes are outside of the scope of this work. In the experiments presented here, the local motion planner uses an RRT* algorithm [8]. Both the Local Motion Planner and Position Controller nodes monitor the robot's true position through Gazebo's internal transform tree. This is done to isolate the localization algorithm's performance without adding complexity. In a real world application where there is no access to a ground truth transform, these localization algorithms would take the place of the transform tree lookup.

3.1.1 Platform & Sensors

The robot models used in Gazebo and ROS2 are defined using Unified Robot Description Format (URDF). This study will use the mobile manipulator model illustrated in Figure 4. It consists of a Husky A200 differential drive mobile platform with a mounted 270° 2-D LiDAR and a manipulator. The manipulator comprises a Universal Robots UR5e collaborative robot arm and a Robotiq 2F-85 end effector. Plugins for Differential Drive, odometry, wheel slip, and LiDAR from Gazebo were leveraged to add model the sensors and actuators in the simulated factory.

Odometry: The `DiffDrive` plugin [7] is used to both publish `nav_msgs/Odometry` messages and receive velocity commands in the form of $u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$. The `WheelSlip` [9] plugin is used to create uncertainty in the odometry to match real world applications. The robot's odometry publishes at 30 Hz. Additionally, `OdometryPublisher` [10] is used to publish the ground truth odometry and transform for results comparison.

LiDAR: The Lidar plugin [11] is used to generate `sensor_msgs/LaserScan` messages. The beam spans $[\phi_{min}, \phi_{max}] = [-2.356, 2.356]$. The transform from the base of the model to the LiDAR

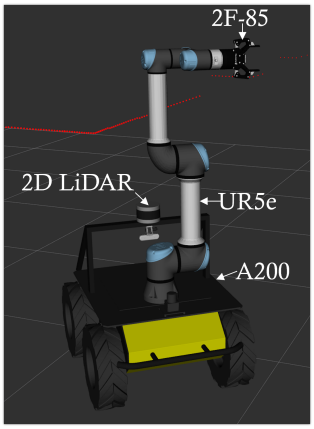


Figure 4: Test robot model.

frame is static and was calculated with the tf2_tools as $t_L = [0.32825, 0.0, 0.0]^T$. The LiDAR scans are preprocessed by the localization algorithms using the same method. Rays that are outside of nominal range, as defined by the `sensor_msgs/LaserScan` message, are thrown out.

3.1.2 Unicycle Kinematics

Since the odometry is produced in the form of $u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$, the kinematic model can be reduced to a Unicycle Kinematic model.

Forward Kinematics: The ros2_controllers page on wheeled kinematics [5] provides the forward kinematics of the unicycle for continuous time shown in Equation 6.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v_{b,x}\cos(\theta) \\ v_{b,x}\sin(\theta) \\ \omega_{b,z} \end{bmatrix} \tag{6}$$

This can be extended to the discrete case as shown in Equation 7:

$$f(\mathbf{x}, \mathbf{u}, \Delta t) = \bar{\mathbf{x}}_t = \begin{bmatrix} x + v\cos(\theta)\Delta t \\ y + v\sin(\theta)\Delta t \\ \theta + \omega\Delta t \end{bmatrix} \tag{7}$$

State Jacobian: The state Jacobian $\mathbf{A}_t$ is calculated by finding the partial derivatives of equation 7 with respect to $\mathbf{x}$.

$$\mathbf{A}_t = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} 1 & 0 & -v\sin(\theta)\Delta t \\ 0 & 1 & v\cos(\theta)\Delta t \\ 0 & 0 & 1 \end{bmatrix} \tag{8}$$

Control Jacobian: The control Jacobian $\mathbf{G}_t$ is calculated by finding the partial derivatives of equation 7 with respect to $\mathbf{u}$.

$$\mathbf{G}_t = \frac{\partial \mathbf{f}}{\partial \mathbf{u}} = \begin{bmatrix} \cos(\theta)\Delta t & 0 \\ \sin(\theta)\Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \tag{9}$$

3.1.3 Odometry Noise Model

Chapter 5 of *Probabilistic Robotics* by Thurn [1] provides algorithms for injecting noise into our odometry sensors. The book argues that command states or measured sensor are subject to noise and that noise can be modeled by a zero-centered random variable with finite variance as Equation 10.

$$\begin{pmatrix} \hat{v} \\ \hat{\omega} \end{pmatrix} = \begin{pmatrix} v \\ \omega \end{pmatrix} + \begin{pmatrix} \sigma_{\alpha_1|v|+\alpha_2|\omega|} \\ \sigma_{\alpha_3|v|+\alpha_4|\omega|} \end{pmatrix} \tag{10}$$

All three algorithms in this study, leverage this concept to model the uncertainty of the odometry. Where $\alpha_1 = 0.10$ represents the translational noise caused by translation. $\alpha_2 = 0.01$ represents the translational noise caused by rotation. $\alpha_3 = 0.01$ represents the rotational noise caused by translation. And $\alpha_4 = 0.20$ represents the rotational noise caused by rotation. These values are used to create noise proportional to the odometry control signal u_t as shown in Equations 11 and 12.

$$\sigma_v = \alpha_1|v_t| + \alpha_2|\omega_t| \tag{11}$$

$$\sigma_\omega = \alpha_3|v_t| + \alpha_4|\omega_t| \tag{12}$$

3.1.4 Map

The open source `octomap_server` [12] is used to generate the static map. A Point Cloud Data (PCD) file was generated of the gazebo world using the open-source `sdf2map` package [13]. The static PCD file is provided to the `octomap_server` at runtime which then publishes an OctoGrid with $0.05m$ resolution and an OccupancyGrid which is a 2-D projection of the OctoGrid. The bottom $0.45m$ of the point cloud is removed before applying the projection to keep the factory floor from occluding the free space.

3.2 Dead Reckoning

Dead Reckoning is simplest algorithm under test. It implements only the prediction step of the Bayes filter shown in equation 4. A full diagram of the implemented dead reckoning algorithm is shown in Figure 5. The prediction step is implemented as a callback method when the Dead Reckoning node receives a `nav_msgs/Odometry` message.

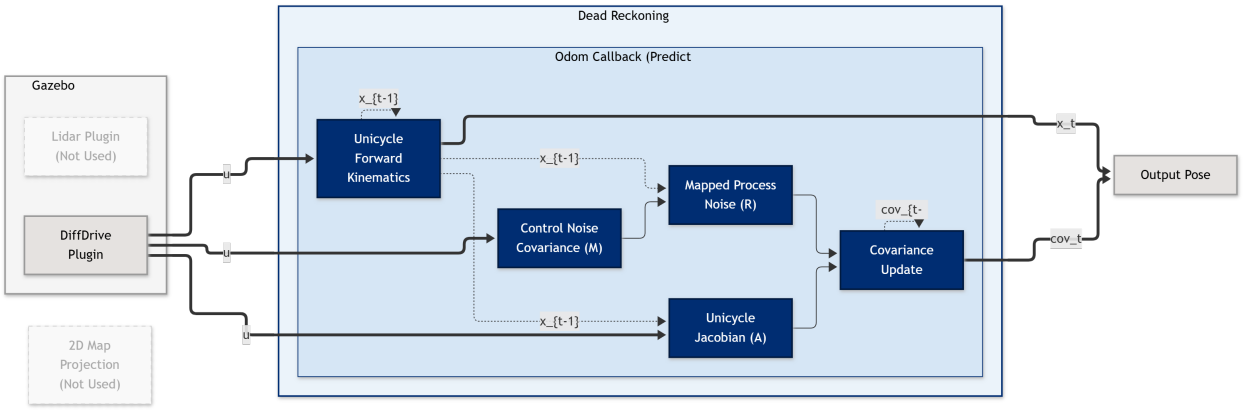


Figure 5: Dead Reckoning Algorithm

3.2.1 Prediction

The algorithm begins by performing the forward kinematics described in Equation 7 to create a prediction of $\bar{\mathbf{x}}_t$. The previous state $\mathbf{x}_{t-1}$ and raw Odometry message $\mathbf{u}_t$ are used to calculate the state Jacobian $\mathbf{A}_t$ shown in equation 8. The control noise covariance matrix M_t can be calculated from the noises calculated in Equations 11 and 12:

$$M_t = \begin{bmatrix} \sigma_v^2 & 0 \\ 0 & \omega_v^2 \end{bmatrix} \tag{13}$$

The mapped process noise R_t is then found using the control Jacobian as shown in Equation 14

$$R_t = G_t M_t G_t^T \Delta t \tag{14}$$

Next, the covariance is updated as shown in Equation 15.

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R \tag{15}$$

At the end of the predict step, the predicted values $\bar{\mathbf{x}}_t$ and $\bar{\Sigma}_t$ are published as a `PoseWithCovarianceStamped` message. The implementation of the Dead Reckoning node and algorithm can be found in Appendix 7.2.

3.3 Extended Kalman Filter (EKF)

The EKF combines the Dead Reckoning prediction step implementation with an implementation of the measurement update step from Equation 5. This study's EKF uses a 2D Iterative Closest Point (ICP) scan matching algorithm to produce

The EKF fuses the Dead Reckoning prediction with the ICP measurement update. Because the system is non-linear, the EKF linearizes the measurement models using Jacobians [1]. An illustration of the EKF node is shown in Figure 6. A KD-Tree is utilized to encode the OccupancyGrid to efficiently find the nearest neighbor in the reference map for each projected LiDAR point.

Because the odometry runs at a higher frequency than the LiDAR does, Odometry messages are buffered in the Odometry callback. In the LiDAR callback, the odom buffer is emptied and the predict step is performed on each reading. Next ICP is performed to determine the observed state z_t and match error $\mathcal{M}$. These values are then fed into the measurement update step shown below.

3.3.1 Augmented Motion Model

During initial testing, there was a yaw-rate error that would build up in the EKF. To mitigate this error, an augmented motion model was designed to include the state and the initial belief for the yaw-rate error which is shown in Equation 16.

$$\xi = \begin{bmatrix} x \\ y \\ \theta \\ s \end{bmatrix} \tag{16}$$

The related covariance matrix is defined as $\mathbf{P} \in \mathbb{R}^{4\times4}$ and the state and covariance are initialized as $\mathbf{x}_0 = [0, 0, 0, 1]^T$ and $\mathbf{P}_0 = \text{diag}(0.1^2, 0.1^2, (5\pi/180)^2, 0.3^2)$. Therefore the filter begins by assuming the odometry is unbiased ($\widehat{s} = 1$), with a standard deviation of 0.3.

Forward Kinematics Substituting true yaw rate $s\omega$ for ω in Equation 7 and apending s.

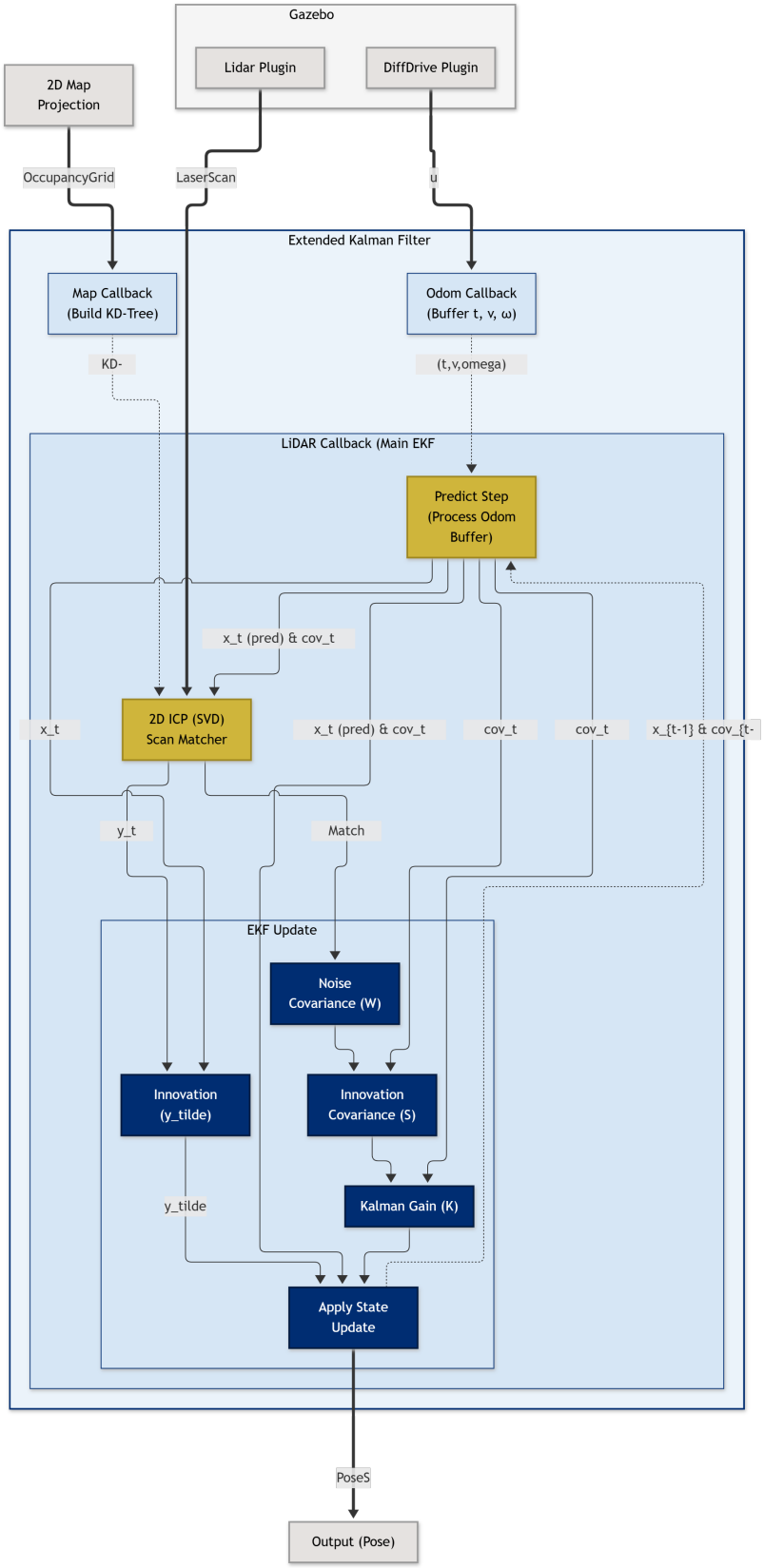


Figure 6: Extended Kalman Filter Update Algorithm

$$f_a(\mathbf{x}, \mathbf{u}, \Delta t) = \bar{\mathbf{x}}_t = \begin{bmatrix} x + v\cos(\theta)\Delta t \\ y + v\sin(\theta)\Delta t \\ \theta + s\omega\Delta t \\ s \end{bmatrix} \tag{17}$$

State Jacobian: The state Jacobian $\mathbf{A}_t$ is calculated by finding the partial derivatives of equation 7 with respect to $\mathbf{x}$.

$$\mathbf{A}_t = \frac{\partial \mathbf{f}_a}{\partial \xi} = \begin{bmatrix} 1 & 0 & -v\sin(\theta)\Delta t & 0 \\ 0 & 1 & v\cos(\theta)\Delta t & 0 \\ 0 & 0 & 1 & \omega\Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{18}$$

Control Jacobian: The control Jacobian $\mathbf{G}_t$ is calculated by finding the partial derivatives of equation 7 with respect to $\mathbf{u}$.

$$\mathbf{G}_t = \frac{\partial \mathbf{f}_a}{\partial \mathbf{u}} = \begin{bmatrix} \cos(\theta)\Delta t & 0 \\ \sin(\theta)\Delta t & 0 \\ 0 & \Delta t \\ 0 & 0 \end{bmatrix} \tag{19}$$

Process noise: The process noise equation 14 is then modified below to include a scaling factor for yaw q_s:

$$R_t = G_t M_t G_t^T \Delta t + \text{diag}(0, 0, 0, q_s^2 \Delta t) \tag{20}$$

3.3.2 Prediction

The EKF prediction step is nearly identical to the Dead Reckoning process described in Section 3.2.1 except the Augmented Motion Model described in Section 3.3.1 was used instead of the unicycle motion model.

3.3.3 Measurement Update

The observed pose $\mathbf{z}_t$ and match error $\mathcal{M}$ come from Iterative Closest Point. The observation matrix is the identify H.

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{21}$$

The innovation is found by Equation 23.

$$\tilde{y}_t = \mathbf{z}_t - \hat{x}_t \tag{22}$$

And the innovation covariance is found by Equation 23.

$$S_t = H\hat{\Sigma}_t H^T + W_t \tag{23}$$

A χ^2 statistical acceptance test is then performed to determine statistical acceptance using $\alpha = 0.95$ and df= 3. If the observation is stastically reasonable, the Kalman gain $\mathbf{K}$ is found by Equation 24

$$K = \hat{\Sigma} H^T S^{-1} \tag{24}$$

Finally the state and covariances are updated as follows:

$$\mathbf{x}_t = \hat{x} + K\tilde{y} \tag{25}$$

$$\Sigma_t = (I - KH)\hat{\Sigma} \tag{26}$$

Scan Matching via ICP and SVD: The ICP algorithm implemented follows point-to-point descriptions from *A Review of Point Cloud Registration Algorithms for Mobile Robotics* [14]. It uses the predicted values $\bar{\mathbf{x}}_t$ and $\bar{\Sigma}_t$, as well as, a KD-Tree built from the OccupancyGrid to output an observed state $\mathbf{z}_t$ and a match error $\mathcal{M}$. The `LaserScan` is processed into $\mathcal{P}$ and $\mathcal{Q}$ which represent the laser points in the localframe and the nearby collision points in the local frame, respectively. First the centroid of the point clouds are found by:

$$\mu_p = \frac{1}{K}\sum_{k=1}^{K} p_k \tag{27}$$

$$\mu_q = \frac{1}{K}\sum_{k=1}^{K} q_k \tag{28}$$

The covariance is found as follows:

$$\mathbf{H} = \sum_{k=1}^{K} (p_k - \mu_p)(q_k - \mu_q)^T \tag{29}$$

Single Value Decomposition (SVD) is then used to find optimal transformation. Let $\mathbf{U}\Lambda\mathbf{V}^T$ be the single value decomponsition H. Then the optimal transformation to minimize the least squared error is shown in Equations 30 and 31:

$$\widehat{R} = \mathbf{V}\mathbf{U}^T \tag{30}$$

$$\widehat{t} = \mu_q - \widehat{R}\mu_p \tag{31}$$

The observed pose $\mathbf{z}_t$ is then found by applying the optimal transformation to $\bar{x}_t$. The implementation of the EKF node and algorithm can be found in Appendix 7.3.

3.4 Particle Filter

The Particle Filter, implemented as Monte Carlo Localization (MCL), represents the belief state using a set of weighted samples and evaluates the Bayes filter Equations 4 and 5 by Monte Carlo simulation [1].

3.4.1 Prediction

The particle filter begins the prediction step by creating noise proportional to the velocity commands for each particle using Equations 11 and 12. It also includes a noise floor value $\sigma_{\omega,\min} = 0.02$ to keep particle spread while the robot is stationary. Random noisy movements are then injected into the particles using a normal distribution.

$$\Delta d^i \sim \mathcal{N}(v\Delta t, \sigma_v^2 \Delta t) \tag{32}$$

$$\Delta \varphi^i \sim \mathcal{N}(\omega\Delta t, \sigma_\omega^2 \Delta t) \tag{33}$$

Those values are then converted into per-particle velocities.

$$v^i = \frac{\Delta d^i}{\Delta t} \tag{34}$$

$$\omega^i = \frac{\Delta \varphi^i}{\Delta t} \tag{35}$$

These velocities are then applied to each particle using the forward kinematics described in Equation 7. An illustration of the PF prediction algorithm can be found in figure 7.

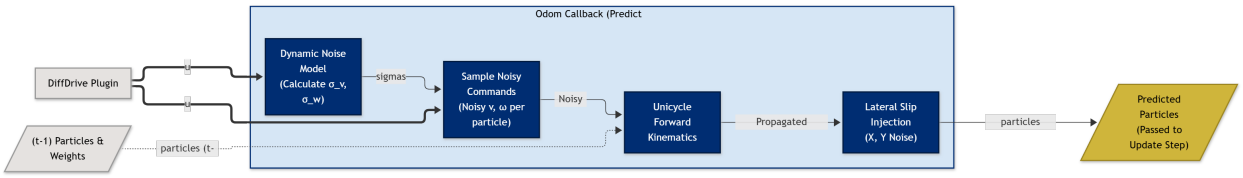


Figure 7: Particle Filter Prediction Algorithm

3.4.2 Measurement Update

The measurement step is illustrated in Figure 7. The weight of each particle is updated using a Likelihood Field sensor model [1]. Rather than extracting a direct pose estimate from the scan like the EKF, the PF evaluates the raw LiDAR ranges against a pre-computed distance transform map found using scipy [15]. For a given particle pose hypothesis, each LiDAR endpoint k is projected into the global map to find its Euclidean distance d_k to the nearest obstacle. Assuming zero-mean Gaussian sensor noise with standard deviation σ_{hit}, the likelihood of a single beam is:

$$p(z_t^k \mid \mathbf{x}_t^{[m]}) \propto \exp\left(-\frac{d_k^2}{2\sigma_{hit}^2}\right) \tag{36}$$

The total weight for particle m is proportional to the product of the likelihoods of all valid beams. To prevent numerical underflow in floating-point operations, this is computed in log-space using the log-sum-exp trick:

$$w_t^{[m]} = \eta \exp\left(\sum_k \log p(z_t^k \mid \mathbf{x}_t^{[m]})\right) \tag{37}$$

where η is a normalization constant ensuring that $\sum_{m=1}^{M} w_t^{[m]} = 1$.

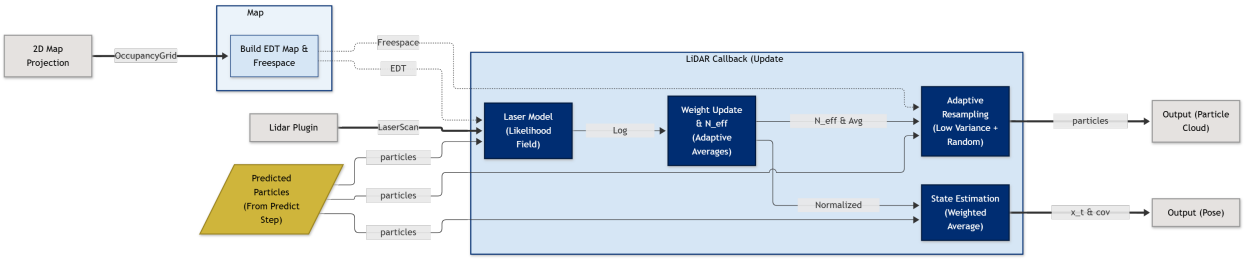


Figure 8: Particle Filter Update Algorithm

3.4.3 Resampling

To prevent particle deprivation, where a majority of the computational effort is spent updating particles with near zero weights, low variance resampling is applied. A new set of M particles is drawn from the current set with replacement, where the probability of drawing particle m is proportional to its weight $w_t^{[m]}$. This focuses the distribution on the most probable regions of the state space. The implementation of the PF node and algorithm can be found in Appendix 7.4.

3.5 Kidnapping

Kidnapping is performed by a seperate `Kidnap` node. The Kidnap node monitors the ground truth state of the robot. When the robot is within $0.3m$ of goal 1 pose, the Kidnap node starts a 15 second timer. When the 15 second timer elaspes, the Kidnap node chooses a random valid position in free space and teleports the robot using a service call to Gazebo. The local motion planner is robust to this and replans from its current pose to the existing goal (i.e. goal 2).

3.6 Metric Collection

Metric collection is performed by the `Metric` node. The Metric node monitors the poses published by the localization nodes and compares them to the ground truth transform. The Metric node compares the localization node poses with the ground truth pose to find the RMSE position and orientation values. The Metric node also subscribes to the runtime topics for each localization nodes which includes runtime since their last estimation.

4 Results

4.1 Algorithm Accuracy

4.2 Runtime Analysis

4.3 Kidnap Recovery

4.4 Particle Count Analysis

5 Conclusion

6 Acknowledgements

This work was completed as part of class world for 605.745 Reasoning Under Uncertainty at Johns Hopkins University taught by Professor Marc Johnson, Ph.D. This work was funded by NASA's Advanced Degree Program.

7 Appendix

7.1 Base Localization Node Implementation

```
#!/usr/bin/env python3

from zlib import crc32
from numpy import ndarray, array, linspace, isinf, isnan, diag, deg2rad, zeros
from numpy.random import default_rng

from rclpy.node import Node
from tf2_ros import Buffer, TransformBroadcaster, TransformListener

from nav_msgs.msg import Odometry, OccupancyGrid
from geometry_msgs.msg import PoseWithCovarianceStamped, PointStamped
from sensor_msgs.msg import LaserScan

from devol_localization.utils import euler_to_quaternion


__author__ = 'Jacob Taylor Cassady'
__email__ = 'jcassad1@jh.edu'


class BaseLocalizationNode(Node):
    def __init__(
        self,
        node_name,
        pose_topic: str = '/devol_drive/pose',
        runtime_topic: str = '/devol_drive/pose/runtime',
    ):
        super().__init__(node_name)

        # Declare Parameters
        self.declare_parameter('odom_topic', '/devol_drive/odom')
        self.declare_parameter('map_topic', '/devol_drive/localization/projected_map')
        self.declare_parameter('scan_topic', '/devol_drive/sensors/lidar2d_0/scan')
        self.declare_parameter('pose_topic', pose_topic)
        self.declare_parameter('runtime_topic', runtime_topic)
        self.declare_parameter('map_frame', 'map')
        self.declare_parameter('odom_frame', 'odom')
        self.declare_parameter('base_frame', 'a200_base_link')
        self.declare_parameter('seed', '31')

        # Retrieve Parameters
        self.odom_topic: str = self.get_parameter('odom_topic').value
        self.map_topic: str = self.get_parameter('map_topic').value
```

```
        self.scan_topic: str = self.get_parameter('scan_topic').value
        self.pose_topic: str = self.get_parameter('pose_topic').value
        self.runtime_topic: str = self.get_parameter('runtime_topic').value
        self.map_frame: str = self.get_parameter('map_frame').value
        self.odom_frame: str = self.get_parameter('odom_frame').value
        self.base_frame: str = self.get_parameter('base_frame').value
        self.seed: int = int(self.get_parameter('seed').value)

        # Shared State and Models
        self.v: float = 0.0
        self.omega: float = 0.0
        self.x_est: ndarray = array([0.0, 0.0, 0.0])
        self.covariance: ndarray = diag([0.1**2, 0.1**2, deg2rad(5.0) ** 2])

        # Proportional Control Noise Parameters
        self.alpha_tv_causedby_tv: float = 0.10
        self.alpha_tv_causedby_rv: float = 0.01
        self.alpha_rv_causedby_tv: float = 0.01
        self.alpha_rv_causedby_rv: float = 0.20

        # Initialize random number generator
        self.rng = default_rng(self.seed)

        self.latest_odom_pose = array([0.0, 0.0, 0.0])
        self.last_map_crc = None
        self.last_scan_time = None

        # Shared Publishers
        self.pub_pose = self.create_publisher(PoseWithCovarianceStamped, self.pose_topic, 10)
        self.runtime_pub = self.create_publisher(PointStamped, self.runtime_topic, 10)

    def odom_callback(self, msg: Odometry):
        """Base callback to track the latest odometry twist."""
        self.v: float = msg.twist.twist.linear.x
        self.omega: float = msg.twist.twist.angular.z
        self.u: ndarray = array([self.v, self.omega])

    def is_map_new(self, msg: OccupancyGrid) -> bool:
        """Computes a CRC32 hash of the map data.
        Returns True if the map is new/changed, False if it is identical."""
        # Convert the map data array to continuous bytes and hash it
        current_crc = crc32(bytes(msg.data))

        if self.last_map_crc == current_crc:
            return False

        self.last_map_crc = current_crc
```

```
        return True

    def get_dt_from_header(self, header):
        """Calculates the time delta (dt) from a message header.
        Updates self.last_scan_time.
        Returns dt (float) if valid, or None if the loop should be skipped."""
        msg_time = header.stamp.sec + (header.stamp.nanosec / 1e9)

        if self.last_scan_time is None:
            self.last_scan_time = msg_time
            return None

        dt = msg_time - self.last_scan_time

        # Prevent massive dt jumps if processing fell way behind
        if dt > 1.0:
            self.get_logger().warn(f'Large dt detected ({dt:.2f}s). Resetting.')
            self.last_scan_time = msg_time
            return None

        self.last_scan_time = msg_time
        return dt

    @staticmethod
    def preprocess_scan(msg: LaserScan):
        """Standardizes LiDAR extraction for all algorithms."""
        angles: ndarray = linspace(msg.angle_min, msg.angle_max, len(msg.ranges))
        ranges: ndarray = array(msg.ranges)

        # Filter out infinite/bad rays
        eps = 0.02
        valid = (
            (ranges > msg.range_min)
            & (ranges < (msg.range_max - eps))
            & ~isinf(ranges)
            & ~isnan(ranges)
        )
        return ranges[valid], angles[valid]

    def get_laser_offset(self):
        """Looks up the physical LiDAR mount offset."""
        return array([0.32825, 0.0, 0.0])

    @staticmethod
    def control_noise_covariance(
        u: ndarray,
        alpha_tv_causedby_tv: float,
```

```
        alpha_tv_causedby_rv: float,
        alpha_rv_causedby_tv: float,
        alpha_rv_causedby_rv: float,
    ) -> ndarray:
        """M"""
        v: float = u[0]
        omega: float = u[1]

        sigma_v: float = alpha_tv_causedby_tv * abs(v) + alpha_tv_causedby_rv * abs(omega) + 0
        sigma_w: float = alpha_rv_causedby_tv * abs(v) + alpha_rv_causedby_rv * abs(omega) + 0

        return diag([sigma_v**2, sigma_w**2])

    @staticmethod
    def update_covariance(A: ndarray, cov: ndarray, R: ndarray) -> ndarray:
        """Sigma"""
        return A @ cov @ A.T + R

    def publish_pose_and_tf_runtime(self, stamp, cov_3x3, start_time: float, end_time: float):
        """Standardized publishing of Pose + Covariance and REP-105 TF."""
        # Publish Pose
        pose_msg = PoseWithCovarianceStamped()
        pose_msg.header.stamp = stamp
        pose_msg.header.frame_id = self.map_frame

        pose_msg.pose.pose.position.x = self.x_est[0]
        pose_msg.pose.pose.position.y = self.x_est[1]
        pose_msg.pose.pose.orientation = euler_to_quaternion(self.x_est[2])

        # Pack 3x3 covariance into ROS 6x6 format
        cov_6x6 = zeros((6, 6))
        cov_6x6[0, 0], cov_6x6[0, 1] = cov_3x3[0, 0], cov_3x3[0, 1]
        cov_6x6[1, 0], cov_6x6[1, 1] = cov_3x3[1, 0], cov_3x3[1, 1]
        cov_6x6[0, 5], cov_6x6[5, 0] = cov_3x3[0, 2], cov_3x3[2, 0]
        cov_6x6[1, 5], cov_6x6[5, 1] = cov_3x3[1, 2], cov_3x3[2, 1]
        cov_6x6[5, 5] = cov_3x3[2, 2]

        pose_msg.pose.covariance = tuple(cov_6x6.flatten())
        self.pub_pose.publish(pose_msg)

        # Publish the runtime as a PointStamped message
        runtime_msg = PointStamped()
        runtime_msg.header.stamp = stamp
        runtime_msg.point.x = end_time - start_time
        self.runtime_pub.publish(runtime_msg)
```

7.2 Dead Reckoning Node Implementation

```
#!/usr/bin/env python3

from typing import Tuple, Optional
from time import perf_counter
from numpy import diag, array, ndarray, sin, cos, zeros, deg2rad

import rclpy
from nav_msgs.msg import Odometry

from devol_localization.base_localization_node import BaseLocalizationNode
from devol_localization.motion_models import UnicycleMotionModel
from devol_localization.utils import wrap_angle


__author__ = 'Jacob Taylor Cassady'
__email__ = 'jcassad1@jh.edu'


class DeadReckoningNode(BaseLocalizationNode):
    def __init__(self):
        super().__init__(
            'dead_reckoning_node',
            pose_topic='/devol_drive/dr_pose',
            runtime_topic='/devol_drive/dr_pose/runtime',
        )

        # State variables.
        self.x_est: ndarray = array([0.0, 0.0, 0.0])  # [x, y, theta]
        self.covariance: ndarray = diag([0.1**2, 0.1**2, deg2rad(5.0) ** 2])
        self.localization_runtime = 0.0

        self.create_subscription(Odometry, self.odom_topic, self.odom_callback, 10)

        self.get_logger().info('Dead Reckoning Node Started!')

    def odom_callback(self, msg: Odometry):
        """Overrides base class implementation"""
        start_time: float = perf_counter()
        dt: Optional[float] = self.get_dt_from_header(msg.header)
        if dt is None:
            self.localization_runtime += perf_counter() - start_time
            return

        self.x_est, self.covariance = DeadReckoningNode._predict(
            prev_x=self.x_est,
```

```
            prev_cov=self.covariance,
            v=msg.twist.twist.linear.x,
            omega=msg.twist.twist.angular.z,
            dt=dt,
            alpha_tv_causedby_tv=self.alpha_tv_causedby_tv,
            alpha_tv_causedby_rv=self.alpha_tv_causedby_rv,
            alpha_rv_causedby_tv=self.alpha_rv_causedby_tv,
            alpha_rv_causedby_rv=self.alpha_rv_causedby_rv,
        )
        self.localization_runtime += perf_counter() - start_time

        # Publish Pose, TF, and Runtime using the base class
        self.publish_pose_and_tf_runtime(
            stamp=msg.header.stamp,
            cov_3x3=self.covariance,
            start_time=0.0,
            end_time=self.localization_runtime,
        )

        # Reset runtime for the next cycle
        self.localization_runtime = 0.0

    @staticmethod
    def mapped_process_noise(x: ndarray, M: ndarray, dt: float) -> ndarray:
        """R"""
        G: ndarray = UnicycleMotionModel.control_jacobian(x=x, dt=dt)
        return G @ M @ G.T * dt

    @staticmethod
    def _predict(
        prev_x: ndarray,
        prev_cov: ndarray,
        v: float,
        omega: float,
        dt: float,
        alpha_tv_causedby_tv: float,
        alpha_tv_causedby_rv: float,
        alpha_rv_causedby_tv: float,
        alpha_rv_causedby_rv: float,
    ) -> Tuple[ndarray, ndarray]:
        """Prediction step. Apply Kinematic model"""
        u: ndarray = array([v, omega])

        # Apply model forward kinematics
        x: ndarray = UnicycleMotionModel.forward(x=prev_x, u=u, noise=zeros(3), dt=dt)
        x: ndarray = wrap_angle(x)
```

```
        # Proportional Control Noise Covariance (M)
        M: ndarray = DeadReckoningNode.control_noise_covariance(
            u=u,
            alpha_tv_causedby_tv=alpha_tv_causedby_tv,
            alpha_tv_causedby_rv=alpha_tv_causedby_rv,
            alpha_rv_causedby_tv=alpha_rv_causedby_tv,
            alpha_rv_causedby_rv=alpha_rv_causedby_rv,
        )

        # Mapped Process Noise (R)
        R: ndarray = DeadReckoningNode.mapped_process_noise(x=x, M=M, dt=dt)

        # Calculate the Jacobian using the previous step.
        A: ndarray = UnicycleMotionModel.state_jacobian(x=prev_x, u=u, dt=dt)

        # Update the covariance
        cov: ndarray = DeadReckoningNode.update_covariance(A=A, cov=prev_cov, R=R)

        return x, cov


def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(DeadReckoningNode())
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

7.3 Extended Kalman Filter Node Implementation

```
#!/usr/bin/env python3

from typing import Tuple
from time import perf_counter
from scipy.spatial import KDTree
from scipy.stats import chi2
from numpy import (
    ndarray,
    array,
    zeros,
    diag,
    deg2rad,
    mean,
    sum,
    sin,
    cos,
    vstack,
    where,
    int8 as np_int8,
    eye,
    arctan2,
    trace,
    sqrt,
    clip,
)
from numpy.linalg import svd, det, inv

import rclpy
from rclpy.qos import DurabilityPolicy, QoSProfile, ReliabilityPolicy

# ROS 2 Messages
from nav_msgs.msg import OccupancyGrid, Odometry
from sensor_msgs.msg import LaserScan

# Import devol localization classes
from devol_localization.base_localization_node import BaseLocalizationNode
from devol_localization.motion_models import UnicycleMotionModel
from devol_localization.utils import wrap_angle


__author__ = 'Jacob Taylor Cassady'
__email__ = 'jcassad1@jh.edu'


# Chi-square threshold for 3 degrees of freedom at 95% confidence
```

```
CHI2_TRESHOLD_3DOF_95CONF: float = 7.815


class EKFLocalizationNode(BaseLocalizationNode):
    def __init__(self):
        super().__init__(
            'ekf_localization_node',
            '/devol_drive/ekf_pose',
            runtime_topic='/devol_drive/ekf_pose/runtime',
        )

        self.map_data = None
        self.W_pose: ndarray = diag([0.05**2, 0.05**2, deg2rad(0.2) ** 2])

        # State variables.
        self.x_est: ndarray = array(
            [0.0, 0.0, 0.0, 1.0]
        )  # [x, y, theta, s], where true yaw rate = s * raw odometry yaw
        self.covariance: ndarray = diag([0.1**2, 0.1**2, deg2rad(5.0) ** 2, 0.3**2])
        self.q_yaw_scale: float = 0.02

        # Time synchronization variables
        self.odom_buffer: list = []
        self.last_time: float = None

        # Total runtime accumulator
        self.localization_runtime: float = 0.0

        # Subscribers / Publishers
        map_qos: QoSProfile = QoSProfile(
            depth=1,
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.TRANSIENT_LOCAL,
        )
        self.create_subscription(OccupancyGrid, self.map_topic, self.map_callback, map_qos)
        self.create_subscription(LaserScan, self.scan_topic, self.scan_callback, 10)
        self.create_subscription(Odometry, self.odom_topic, self.odom_callback, 10)

        self.get_logger().info('EKF Localization Node Started!')

    def odom_callback(self, msg: Odometry):
        """Base callback to track the latest odometry twist."""
        start_time: float = perf_counter()
        # Convert ROS timestamp to a float in seconds
        stamp = msg.header.stamp
        t_odom = stamp.sec + stamp.nanosec * 1e-9
        v = msg.twist.twist.linear.x
```

```
        omega = msg.twist.twist.angular.z

        # Add to the buffer
        self.odom_buffer.append((t_odom, v, omega))
        self.localization_runtime += perf_counter() - start_time

    def map_callback(self, msg: OccupancyGrid):
        """Extracts obstacle coordinates and builds KDTree."""
        start_time: float = perf_counter()

        if not self.is_map_new(msg):
            self.localization_runtime += perf_counter() - start_time
            return

        grid: ndarray = array(msg.data, dtype=np_int8).reshape((msg.info.height, msg.info.widt

        # Find the row/col indices of all cells that are likely obstacles
        obstacle_y, obstacle_x = where(grid > 50)

        # Convert indices to real-world metric coordinates
        map_pts_x: ndarray = msg.info.origin.position.x + (obstacle_x + 0.5) * msg.info.resolu
        map_pts_y: ndarray = msg.info.origin.position.y + (obstacle_y + 0.5) * msg.info.resolu

        # Stack into an Nx2 array of obstacle points
        map_points: ndarray = vstack((map_pts_x, map_pts_y)).T

        # Build the KDTree for the ICP algorithm
        self.map_data: dict = {'points': map_points, 'kdtree': KDTree(map_points)}
        self.get_logger().info(
            f'Map received. Built KDTree with {len(map_points)} obstacle points.'
        )

        self.localization_runtime += perf_counter() - start_time

    def scan_callback(self, msg: LaserScan):
        """The main loop of the EKF."""
        start_time: float = perf_counter()

        if self.map_data is None:
            self.localization_runtime += perf_counter() - start_time
            return  # Wait for map to load

        # Get exact timestamp of the laser scan
        stamp = msg.header.stamp
        t_scan = stamp.sec + stamp.nanosec * 1e-9

        # Initialize time tracker on the very first scan
```

```
if self.last_time is None:
    self.last_time = t_scan
    self.localization_runtime += perf_counter() - start_time
    return

# Process all buffered odom messages stamped AT or BEFORE the scan time
x_pred, cov_pred = self.x_est.copy(), self.covariance.copy()
current_v = 0.0
current_omega = 0.0
while self.odom_buffer and self.odom_buffer[0][0] <= t_scan:
    t_odom, v, omega = self.odom_buffer.pop(0)

    dt = t_odom - self.last_time
    if dt > 0:
        x_pred, cov_pred = EKFLocalizationNode._predict(
            prev_x=x_pred,
            prev_cov=cov_pred,
            v=v,
            omega=omega,
            dt=dt,
            q_yaw_scale=self.q_yaw_scale,
            alpha_tv_causedby_tv=self.alpha_tv_causedby_tv,
            alpha_tv_causedby_rv=self.alpha_tv_causedby_rv,
            alpha_rv_causedby_tv=self.alpha_rv_causedby_tv,
            alpha_rv_causedby_rv=self.alpha_rv_causedby_rv,
        )
        self.last_time = t_odom

    # Keep track of the latest velocities for the partial step
    current_v = v
    current_omega = omega

# Add one final partial step exactly up to the scan timestamp
dt_partial = t_scan - self.last_time
if dt_partial > 0:
    x_pred, cov_pred = EKFLocalizationNode._predict(
        prev_x=x_pred,
        prev_cov=cov_pred,
        v=current_v,
        omega=current_omega,
        dt=dt_partial,
        q_yaw_scale=self.q_yaw_scale,
        alpha_tv_causedby_tv=self.alpha_tv_causedby_tv,
        alpha_tv_causedby_rv=self.alpha_tv_causedby_rv,
        alpha_rv_causedby_tv=self.alpha_rv_causedby_tv,
        alpha_rv_causedby_rv=self.alpha_rv_causedby_rv,
    )
```

```
            self.last_time = t_scan

        # update step (Using LaserScan ICP)
        z, is_valid, match_error = EKFLocalizationNode.scan_match(
            msg=msg,
            x_pred=x_pred,
            cov_pred=cov_pred,
            map_data=self.map_data,
            laser_offset=self.get_laser_offset(),
        )

        if is_valid:
            self.x_est, self.covariance, accepted = EKFLocalizationNode._update(
                x_pred=x_pred, cov_pred=cov_pred, W_pose=self.W_pose, z=z, match_error=match_e
            )

            if not accepted:
                self.get_logger().debug('ICP match rejected by Mahalanobis check.')
        else:
            self.x_est = x_pred
            self.covariance = cov_pred

        # Publish state
        self.localization_runtime += perf_counter() - start_time
        self.publish_pose_and_tf_runtime(
            stamp=msg.header.stamp,
            cov_3x3=self.covariance,
            start_time=0.0,
            end_time=self.localization_runtime,
        )

        # Reset runtime for the next cycle
        self.localization_runtime = 0.0

    @staticmethod
    def scan_match(
        msg: LaserScan,
        x_pred: ndarray,
        cov_pred: ndarray,
        map_data: dict,
        laser_offset: ndarray,
        max_iter: int = 200,
        tolerance: float = 1e-4,
    ):
        """2D Iterative Closest Point using SVD."""
        # Filter out invalid rays
        ranges, angles = EKFLocalizationNode.preprocess_scan(msg=msg)
```

```
if len(ranges) < 10:
    return array([0.0, 0.0, 0.0, 1.0]), False, 0.0  # Not enough valid points

# Nx2 array of local laser points
laser_pts: ndarray = vstack((ranges * cos(angles), ranges * sin(angles))).T

R_offset: ndarray = array(
    [
        [cos(laser_offset[2]), -sin(laser_offset[2])],
        [sin(laser_offset[2]), cos(laser_offset[2])],
    ]
)
local_pts: ndarray = (R_offset @ laser_pts.T).T + laser_offset[:2]

z: ndarray = x_pred[:3].copy()
prev_error: float = float('inf')

# Calculate adaptive correspondence gate based on positional covariance
pos_variance = trace(cov_pred[:2, :2])
#  3-sigma radius, clamped between 0.5m and 2.0m
search_radius: float = float(clip(3.0 * sqrt(pos_variance), 0.5, 2.0))

for _ in range(max_iter):
    # Transform local scan points into the global map frame
    theta: float = z[2]
    R_curr: ndarray = array([[cos(theta), -sin(theta)], [sin(theta), cos(theta)]])
    global_pts: ndarray = (R_curr @ local_pts.T).T + z[:2]

    # Find the closest Map points (Q) for every Global Scan point (P)
    distances, indices = map_data['kdtree'].query(global_pts)

    # Reject outlier points using the adaptive search radius
    good_matches: ndarray = distances < search_radius
    if sum(good_matches) < 10:
        break  # Lost tracking

    # SVD-Based Alignment (Arun et al., 1987 / Umeyama, 1991)
    P: ndarray = global_pts[good_matches]
    Q: ndarray = map_data['points'][indices[good_matches]]

    # Compute Centroids
    centroid_P: ndarray = mean(P, axis=0)
    centroid_Q: ndarray = mean(Q, axis=0)

    # Center the points around the origin
    P_centered: ndarray = P - centroid_P
```

```
            Q_centered: ndarray = Q - centroid_Q

            # Compute Cross-Covariance Matrix (H)
            H: ndarray = P_centered.T @ Q_centered

            # Singular Value Decomposition
            U, _, Vt = svd(H)

            # Optimal Rotation (R_delta)
            R_delta: ndarray = Vt.T @ U.T
            # Ensure it is a pure rotation, not a reflection
            if det(R_delta) < 0:
                Vt[-1, :] *= -1
                R_delta = Vt.T @ U.T

            # Optimal Translation (t_delta)
            t_delta: ndarray = centroid_Q - R_delta @ centroid_P

            # Apply the delta transformation to current pose guess
            z[:2] = R_delta @ z[:2] + t_delta
            z[2] += arctan2(R_delta[1, 0], R_delta[0, 0])

            # Check for convergence based on Mean Squared Error
            mean_error: float = mean(distances[good_matches])
            if abs(prev_error - mean_error) < tolerance:
                break
            prev_error = mean_error

        # Format final pose
        z = wrap_angle(z)

        is_valid: bool = EKFLocalizationNode.inlier_fraction_check(
            current_pose=z, local_pts=local_pts, map_data=map_data
        )

        return z, is_valid, prev_error

    @staticmethod
    def inlier_fraction_check(
        current_pose: ndarray,
        local_pts: ndarray,
        map_data: dict,
        distance_threshold: float = 0.15,
        valid_threshold: float = 0.8,
    ) -> bool:
        # Project scan points to the global map frame using the pose
        final_theta: float = current_pose[2]
```

```
        R_final: ndarray = array(
            [[cos(final_theta), -sin(final_theta)], [sin(final_theta), cos(final_theta)]]
        )
        final_global_pts: ndarray = (R_final @ local_pts.T).T + current_pose[:2]

        # Query distances to the nearest map points.
        final_distances, _ = map_data['kdtree'].query(final_global_pts)

        # Calculate the fraction of points within distance_threshold
        inliers = sum(final_distances <= distance_threshold)
        inlier_fraction = float(inliers) / len(final_global_pts)

        # Consider the match "valid" if within valid_threshold
        return inlier_fraction >= valid_threshold

    @staticmethod
    def mapped_process_noise(x: ndarray, M: ndarray, dt: float, q_yaw_scale: float) -> ndarray
        """R"""
        theta: float = x[2]
        G = array([[cos(theta), 0.0], [sin(theta), 0.0], [0.0, 1.0], [0.0, 0.0]])
        R: ndarray = G @ M @ G.T * dt
        R[3, 3] += q_yaw_scale**2 * dt
        return R

    @staticmethod
    def _predict(
        prev_x: ndarray,
        prev_cov: ndarray,
        v: float,
        omega: float,
        dt: float,
        q_yaw_scale: float,
        alpha_tv_causedby_tv: float,
        alpha_tv_causedby_rv: float,
        alpha_rv_causedby_tv: float,
        alpha_rv_causedby_rv: float,
    ) -> Tuple[ndarray, ndarray]:
        """Prediction step. Apply Kinematic model"""
        # Package the inputs for the motion model
        s: float = prev_x[3]
        u: ndarray = array([v, s * omega])

        # Apply model forward kinematics
        x: ndarray = prev_x.copy()
        x[:3] = UnicycleMotionModel.forward(x=prev_x[:3], u=u, noise=zeros(3), dt=dt)
        x: ndarray = wrap_angle(x)
```

```
        # Proportional Control Noise Covariance (M)
        M: ndarray = EKFLocalizationNode.control_noise_covariance(
            u=u,
            alpha_tv_causedby_tv=alpha_tv_causedby_tv,
            alpha_tv_causedby_rv=alpha_tv_causedby_rv,
            alpha_rv_causedby_tv=alpha_rv_causedby_tv,
            alpha_rv_causedby_rv=alpha_rv_causedby_rv,
        )

        # Mapped Process Noise (R)
        R: ndarray = EKFLocalizationNode.mapped_process_noise(
            x=prev_x, M=M, dt=dt, q_yaw_scale=q_yaw_scale
        )

        # Calculate the Jacobian using the previous step.
        A: ndarray = eye(4)
        A[:3, :3] = UnicycleMotionModel.state_jacobian(x=prev_x[:3], u=u, dt=dt)
        A[2, 3] = omega * dt

        # Update the covariance
        cov: ndarray = EKFLocalizationNode.update_covariance(A=A, cov=prev_cov, R=R)

        return x, cov

    @staticmethod
    def _update(
        x_pred: ndarray, cov_pred: ndarray, W_pose: ndarray, z: ndarray, match_error: float
    ) -> Tuple[ndarray, ndarray, bool]:
        error_scale: float = max(1.0, (match_error / 0.05) ** 2)
        W_dynamic: ndarray = W_pose * error_scale

        # Observation matrix (H is identity because we observe the direct [x, y, theta, s] pose
        H: ndarray = eye(3, 4)

        # Innovation covariance (S)
        S: ndarray = H @ cov_pred @ H.T + W_dynamic

        # Innovation (y_tilde) is the difference between observation and prediction
        y_tilde: ndarray = wrap_angle(z - x_pred[:3])

        # Statistical Acceptance Test
        mahalanobis_sq = y_tilde.T @ inv(S) @ y_tilde

        if mahalanobis_sq > chi2.ppf(0.95, 3):
            # The observation is statistically absurd given current belief.
            # Reject the update and keep prior estimate.
            return x_pred, cov_pred, False
```

```
        # Kalman Gain (K)
        K: ndarray = cov_pred @ H.T @ inv(S)

        # Update the state estimate and shrink the covariance
        x_update: ndarray = x_pred + K @ y_tilde
        x_update[3] = clip(x_update[3], 0.5, 1.2)
        cov_update: ndarray = (eye(4) - K @ H) @ cov_pred

        # Wrap heading to [-pi, pi] to prevent runaway angle values over time
        x_update = wrap_angle(x_update)

        return x_update, cov_update, True


def main(args=None):
    rclpy.init(args=args)
    node = EKFLocalizationNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

7.4 Particle Filter Node Implementation

```
#!/usr/bin/env python3

from typing import Optional
from time import perf_counter
from scipy.ndimage import distance_transform_edt
from numpy import (
    ndarray,
    array,
    cov,
    float32 as np_float32,
    ones_like,
    ones,
    searchsorted,
    cumsum,
    average,
    arctan2,
    cos,
    sin,
    copy as np_copy,
    arange,
    exp as np_exp,
    max as np_max,
    zeros,
    vstack,
    where,
    pi,
    mean as np_mean,
)

import rclpy
from rclpy.qos import DurabilityPolicy, QoSProfile, ReliabilityPolicy

# ROS 2 Messages
from nav_msgs.msg import OccupancyGrid, Odometry
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import PoseArray, Pose

# Import devol localization classes
from devol_localization.base_localization_node import BaseLocalizationNode
from devol_localization.laser_model import LaserModel
from devol_localization.motion_models import UnicycleMotionModel
from devol_localization.utils import euler_to_quaternion


__author__ = 'Jacob Taylor Cassady'
```

```
__email__ = 'jcassad1@jh.edu'


class ParticleFilterNode(BaseLocalizationNode):
    def __init__(self):
        super().__init__(
            'particle_filter_node',
            '/devol_drive/pf_pose',
            runtime_topic='/devol_drive/pf_pose/runtime',
        )

        self.declare_parameter('num_particles', '500')

        self.map_dict = None
        self.num_particles: int = int(self.get_parameter('num_particles').value)

        # Initialize particles clustered around starting origin
        self.particles: ndarray = self.rng.multivariate_normal(
            self.x_est, self.covariance, self.num_particles
        )
        self.weights: ndarray = ones(self.num_particles) / self.num_particles

        # Adaptive injection moving averages
        self.slow_avg_lik = 0.0
        self.fast_avg_lik = 0.0

        # Total runtime accumulator
        self.localization_runtime = 0.0

        # Subscribers / Publishers
        map_qos: QoSProfile = QoSProfile(
            depth=1,
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.TRANSIENT_LOCAL,
        )
        self.create_subscription(OccupancyGrid, self.map_topic, self.map_callback, map_qos)
        self.create_subscription(LaserScan, self.scan_topic, self.scan_callback, 1)
        self.create_subscription(Odometry, self.odom_topic, self.odom_callback, 10)

        self.pub_particles = self.create_publisher(PoseArray, self.pose_topic + '_cloud', 10)

        self.get_logger().info('Particle Filter Node Started!')

    def odom_callback(self, msg: Odometry):
        """Base callback to track the latest odometry twist."""
        start_time: float = perf_counter()
        self.v: float = msg.twist.twist.linear.x
```

```
        self.omega: float = msg.twist.twist.angular.z
        self.u: ndarray = array([self.v, self.omega])

        dt: Optional[float] = self.get_dt_from_header(msg.header)
        if dt is None:
            return

        self.particles = self._predict(dt=dt)
        self.localization_runtime += perf_counter() - start_time

    def map_callback(self, msg: OccupancyGrid):
        """Converts ROS OccupancyGrid into a distance transform map for the Likelihood field""
        start_time: float = perf_counter()

        if not self.is_map_new(msg):
            self.localization_runtime += perf_counter() - start_time
            return

        # Convert 1D array to 2D numpy array
        grid: ndarray = array(msg.data, dtype=np_float32).reshape(
            (msg.info.height, msg.info.width)
        )

        # Create binary map where obstacle = 0, free space = 1
        binary_map: ndarray = ones_like(grid)
        binary_map[grid > 50] = 0

        # Calculate Euclidean distance to nearest obstacle (in pixels)
        distance_grid: ndarray = distance_transform_edt(binary_map)
        edt_map: ndarray = distance_grid * msg.info.resolution

        # Pre-compute free space coordinates for the escape hatch
        free_y, free_x = where(binary_map == 1)
        res: float = msg.info.resolution
        ox: float = msg.info.origin.position.x
        oy: float = msg.info.origin.position.y

        # Convert pixel indices to real-world coordinates
        free_world_x: ndarray = free_x * res + ox + (res / 2.0)
        free_world_y: ndarray = free_y * res + oy + (res / 2.0)

        # Store in dict to pass to LaserModel easily
        self.map_dict: dict = {
            'grid': edt_map,
            'info': msg.info,
            'free_x': free_world_x,
            'free_y': free_world_y,
```

```
        }
        self.get_logger().info('Map received and Euclidean Distance Transform computed.')
        self.localization_runtime += perf_counter() - start_time

    def scan_callback(self, msg: LaserScan):
        """The main loop of the PF: computes dt, downsamples scan, runs PF, and publishes resul
        start_time: float = perf_counter()

        # Wait for map and ensure odometry has ran at least once.
        if self.map_dict is None:
            self.localization_runtime += perf_counter() - start_time
            return

        z_dists, z_thetas = self.preprocess_scan(msg=msg)

        if len(z_dists) == 0:
            self.localization_runtime += perf_counter() - start_time
            return  # No valid measurements

        # Run the Particle Filter using Odometry values and the Unicycle model
        self.particles, self.x_est = self._update(z_dists=z_dists, z_thetas=z_thetas)
        self.localization_runtime += perf_counter() - start_time

        # Publish state and particles
        self.publish_state(msg.header.stamp)

    def publish_state(self, stamp):
        """Publishes the estimated pose and the particle cloud for RViz"""
        cov_3x3: ndarray = cov(self.particles, rowvar=False)

        # Publish state
        self.publish_pose_and_tf_runtime(
            stamp=stamp, cov_3x3=cov_3x3, start_time=0.0, end_time=self.localization_runtime
        )

        # Publish Particle Cloud
        cloud_msg: PoseArray = PoseArray()
        cloud_msg.header.stamp = stamp
        cloud_msg.header.frame_id = self.map_frame
        for p in self.particles:
            pose = Pose()
            pose.position.x = p[0]
            pose.position.y = p[1]
            pose.orientation = euler_to_quaternion(p[2])
            cloud_msg.poses.append(pose)

        self.pub_particles.publish(cloud_msg)
```

```
        # Reset runtime for the next cycle
        self.localization_runtime = 0.0

    def _predict(self, dt: float) -> ndarray:
        ## Apply the motion model to the samples (Prediction Step)
        # Create noise proportional to the velocity commands for each particle
        sigma_v: float = self.alpha_tv_causedby_tv * abs(self.v) + self.alpha_tv_causedby_rv *
            self.omega
        )
        sigma_w: float = self.alpha_rv_causedby_tv * abs(self.v) + self.alpha_rv_causedby_rv *
            self.omega
        )

        # Add a small floor (0.02) to heading noise to maintain diversity when stopped
        sigma_w = max(0.02, sigma_w)

        # Draw noise applied to the distance and heading moved using sqrt(dt)
        sqrt_dt = dt**0.5

        # Calculate nominal distances and headings moved in this timestep
        dist_moved = self.v * dt
        heading_moved = self.omega * dt

        # Vectorized generation of noisy movements (std_dev = sigma * sqrt(dt))
        noisy_dists = self.rng.normal(dist_moved, sigma_v * sqrt_dt, self.num_particles)
        noisy_headings = self.rng.normal(heading_moved, sigma_w * sqrt_dt, self.num_particles)

        # Convert back to effective velocities since UnicycleMotionModel expects velocity input
        noisy_vs = noisy_dists / dt
        noisy_ws = noisy_headings / dt

        # Apply the forward model using the noisy velocities for each particle
        new_particles: ndarray = array(
            [
                UnicycleMotionModel.forward(
                    self.particles[i],
                    array([noisy_vs[i], noisy_ws[i]]),
                    zeros(3),
                    dt,
                )
                for i in range(self.num_particles)
            ]
        )

        # Inject independent X and Y lateral slip noise directly to the array
        new_particles[:, 0] += self.rng.normal(0.0, 0.005, self.num_particles)
```

```
        new_particles[:, 1] += self.rng.normal(0.0, 0.005, self.num_particles)

        return new_particles

    def _update(self, z_dists: list, z_thetas: list):
        ## Update
        # Calculate where the laser actually is for each newly predicted particle
        laser_poses: ndarray = np_copy(self.particles)
        laser_offset: ndarray = self.get_laser_offset()
        # Shift X and Y based on the laser's physical mount and the robot's current yaw
        laser_poses[:, 0] += laser_offset[0] * cos(laser_poses[:, 2]) - laser_offset[1] * sin(
            laser_poses[:, 2]
        )
        laser_poses[:, 1] += laser_offset[0] * sin(laser_poses[:, 2]) + laser_offset[1] * cos(
            laser_poses[:, 2]
        )
        # Shift Yaw
        laser_poses[:, 2] += laser_offset[2]

        # Computes log probabilities
        log_weights = array(
            [
                LaserModel.likelihood_field_range_finder(
                    z_dists=z_dists, z_thetas=z_thetas, x=particle, map_dict=self.map_dict, sto
                )
                for particle in laser_poses
            ],
            dtype=float,
        )

        # shift all weights so their highest value is 0.
        max_log_weights: ndarray = np_max(log_weights)
        step_weights: ndarray = np_exp(log_weights - max_log_weights)

        # Calculate raw likelihood average for the adaptive injection logic
        avg_raw_likelihood = np_mean(np_exp(log_weights / len(z_dists)))

        # Initialize running averages on the very first update
        if self.slow_avg_lik == 0.0:
            self.slow_avg_lik = avg_raw_likelihood
            self.fast_avg_lik = avg_raw_likelihood

        # Update exponential moving averages (standard AMCL alpha values)
        self.slow_avg_lik += 0.001 * (avg_raw_likelihood - self.slow_avg_lik)
        self.fast_avg_lik += 0.1 * (avg_raw_likelihood - self.fast_avg_lik)

        # Multiply the new step weights into the running weight accumulator
```

```
self.weights *= step_weights

# Prevent division by zero
self.weights += 1e-40
# normalize (\eta)
self.weights /= sum(self.weights)

# Calculate Effective Sample Size (N_eff)
n_eff = 1.0 / sum(self.weights**2)

# Estimate the state based on the weighted average of particles
x_est: ndarray = average(self.particles, axis=0, weights=self.weights)

# Wrap angle
x_est[2] = arctan2(
    average(sin(self.particles[:, 2]), weights=self.weights),
    average(cos(self.particles[:, 2]), weights=self.weights),
)

# Conditional Resampling: Only resample if N_eff drops below 50%
if n_eff < (self.num_particles / 2.0):
    # Calculate adaptive escape fraction
    escape_fraction = 0.0
    if self.slow_avg_lik > 0 and self.fast_avg_lik < self.slow_avg_lik:
        # The fast average dropped below the slow average; spike injection rate.
        escape_fraction = 1.0 - (self.fast_avg_lik / self.slow_avg_lik)

    # Cap the maximum injection
    escape_fraction = max(0.0, min(0.05, escape_fraction))

    num_random = int(self.num_particles * escape_fraction)
    num_normal = self.num_particles - num_random
    normal_particles: ndarray = self.low_variance_resample(
        self.particles, self.weights, num_samples=num_normal
    )

    # Draw fresh random particles uniformly from map freespace
    free_x = self.map_dict.get('free_x', array([]))
    free_y = self.map_dict.get('free_y', array([]))

    if num_random > 0:
        if len(free_x) > 0:
            rand_indices = self.rng.choice(len(free_x), size=num_random, replace=True)
            random_particles = zeros((num_random, 3))
            random_particles[:, 0] = free_x[rand_indices]
            random_particles[:, 1] = free_y[rand_indices]
            random_particles[:, 2] = self.rng.uniform(-pi, pi, num_random)
```

```
                else:
                    # Safe fallback if map processing missed freespace coordinates
                    random_particles = self.rng.multivariate_normal(
                        x_est, cov(self.particles, rowvar=False), num_random
                    )

                    # Combine both groups to form the new generation
                    new_particles: ndarray = vstack((normal_particles, random_particles))
            else:
                new_particles = normal_particles

            # Reset weights to uniform after a resample event
            self.weights = ones(self.num_particles) / self.num_particles
        else:
            new_particles = self.particles

        return new_particles, x_est

    def low_variance_resample(
        self, particles: ndarray, weights: ndarray, num_samples: Optional[int] = None
    ) -> ndarray:
        N = len(particles) if num_samples is None else num_samples

        # Generate a single random start point between 0 and 1/N
        step = 1.0 / N
        pointers = self.rng.uniform(0, step) + arange(N) * step

        # Find where those evenly spaced pointers land in the cumulative weight array
        cumulative_sum = cumsum(weights)
        indexes = searchsorted(cumulative_sum, pointers)

        return particles[indexes]


def main(args=None):
    rclpy.init(args=args)
    node = ParticleFilterNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
```

```
main()
```

References

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[3] G. Pardo-Castellote, "Omg data-distribution service: Architectural overview," in *23rd International Conference on Distributed Computing Systems Workshops, 2003. Proceedings.* IEEE, 2003, pp. 200–206.

[4] W. Burgard, D. Fox, and S. Thrun, "Markov localization for mobile robots in dynamic environments," *CoRR*, vol. abs/1106.0222, 2011. [Online]. Available: https://doi.org/10.48550/arXiv.1106.0222

[5] ROS 2 Control, *Wheeled Mobile Robot Kinematics*, Open Source Robotics Foundation, accessed: 2026-10-19. [Online]. Available: https://control.ros.org/master/doc/ros2_controllers/doc/mobile_robot_kinematics.html

[6] A. Doucet, S. Godsill, and C. Andrieu, "On sequential monte carlo sampling methods for bayesian filtering," *Statistics and Computing*, vol. 10, pp. 197–208, 2000.

[7] Gazebo, "DiffDrive Class Reference," https://gazebosim.org/api/sim/8/classgz_1_1sim_1_1systems_1_1DiffDrive.html, gazebo Sim 10 API Documentation, accessed October 31, 2026.

[8] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The international journal of robotics research*, vol. 30, no. 7, pp. 846–894, 2011.

[9] Gazebo, "WheelSlip Class Reference," https://gazebosim.org/api/sim/10/classgz_1_1sim_1_1systems_1_1WheelSlip.html, gazebo Sim 10 API Documentation, accessed October 31, 2026.

[10] ——, "OdometryPublisher Class Reference," https://gazebosim.org/api/sim/10/classgz_1_1sim_1_1systems_1_1OdometryPublisher.html, gazebo Sim 10 API Documentation, accessed October 31, 2026.

[11] ——, "Lidar Class Reference," https://gazebosim.org/api/sim/10/classgz_1_1sim_1_1Lidar.html, gazebo Sim 10 API Documentation, accessed October 31, 2026.

[12] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.

[13] atinfinity, "sdf2map," https://github.com/atinfinity/sdf2map.

[14] F. Pomerleau, F. Colas, and R. Siegwart, "A review of point cloud registration algorithms for mobile robotics," *Foundations and trends® in Robotics*, vol. 4, no. 1, pp. 1–104, 2015.

[15] SciPy, "distance_transform_edt," https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.distance_transform_edt.html.